

GomMold

An AI-based Application for Mold Detection and Prevention

Aina Nur Insyeerah
Binti Abd Rahman
dept. Information Systems
Hanyang University
Seoul, Republic of Korea
syecera@hanyang.ac.kr

Nur Ain Syafiqah
Binti Rahiman
dept. Information Systems
Hanyang University
Seoul, Republic of Korea
ainrahiman@hanyang.ac.kr

Nur Shaqeerah
Binti Shamsul Anuar
dept. Information Systems
Hanyang University
Seoul, Republic of Korea
qeerashamsul@hanyang.ac.kr

Yumni Karmila
Binti Yunus
dept. Information Systems
Hanyang University
Seoul, Republic of Korea
yumnikarmila@hanyang.ac.kr

Abstract—Mold growth is a common but often overlooked problem in humid environments, leading to health risks, unpleasant odors, and structural damage in homes. GomMold is an application that introduces an AI-powered system to detect and prevent household mold that enables users to identify mold through photos. Traditional mold testing methods that rely on manual visual inspection are often time-consuming, subjective, and prone to error. By utilizing advancements in artificial intelligence (AI) and machine learning, this project applies computer vision and pattern recognition techniques to automate mold detection with high precision.

The system integrates AI-driven image analysis and conversational feedback to provide users with accurate detection, and practical cleaning and prevention advice. In the future, there is potential for GomMold to be integrated into smart home devices such as humidifiers or air purifiers for automated humidity control and prevention advice.

Index Terms—Mold detection, indoor air quality, computer vision, deep learning, environmental monitoring, AI health technology, image classification, preventive maintenance

I. ROLE ASSIGNMENT

The distribution of project roles is summarized in Table 1 to optimize workflow within a small team.

Roles	Name	Task description & etc.
User	Nur Ain Syafiqah	The primary role is to experience and validate the AI mold detection model on application from a non-experts' perspective. Also, play the role to define real-world user scenarios by detecting and inspecting mold on various surfaces such as walls, furniture, and bathrooms.
AI Developer	Nur Shaqeerah	Responsible for artificial intelligence mold detection from developing, training and optimizing the image recognition and classification model, to accurately detect and identify the mold from user-uploaded photos.

Software developer

Aina Nur Insyeerah

Responsible for building the technical framework that supports the AI to integrate in this project from designing the frontend, backend system and database.

Development manager

Yumni Karmila

Oversee and coordinate all aspects of the AI mold detection app's development project. Additionally, responsible for timeline management, project planning and making sure the frontend, backend and AI integrate smoothly without any problems.

II. INTRODUCTION

A. Motivation

Moving to South Korea as foreign students from Malaysia, one challenge that surprised us the most was not the food or the culture, it was the persistent spots of mold appearing in our homes. Especially during the summer season where it's humid and raining all day long and in my case even during winter because of the condensation from the window dripped into my wallpaper, leaving it all wet. Unlike South Korea, we do not experience four seasons or drastic temperature changes in Malaysia. Additionally, house ventilation systems in South Korea also differ from those in Malaysia, which further affects how mold develops indoors. At first, we did not know whether these spots were actually mold or just dirt clumping up, as we were unfamiliar with it. It was only when the situation worsened that we realized that it was too late. If we had recognized that it was mold earlier, we could have prevented it from getting severe. We realized that we are not the only ones facing this problem. Students living in rented houses, poorly ventilated spaces, and even long-term residents in Korea often struggle to identify mold growth. It is not just a lack of knowledge, many people, even those who have lived in Korea and experienced it for years, have difficulty recognizing mold in their homes.

Our motivation for this project comes from this experience. We wanted to create a tool that helps anyone to detect

and manage mold confidently. We aim to develop an app that is user-friendly and allows everyone to easily identify mold in their homes within seconds. By combining AI with accessible technology, we aim to simplify a problem that is often overlooked but can have serious health and structural consequences. With this project, users can identify whether it is mold from photos taken from their phones and receive clear cleaning advice all in one place. Our goal is not just to make mold detection easier, but to empower people to live in healthier, safer homes, turning an unfamiliar and potentially hazardous issue into something manageable and understandable. We are also motivated by the potential of AI technology to make this process intuitive and user-friendly. By leveraging the advanced features of AI, we can quickly identify mold, all without the need for professional tools or expertise. Through this project, we hope to demonstrate how AI can transform a common household problem into an easy, and intelligent solution, helping many people live safer and healthier lives

B. Problem Statement

1) Difficulty Identifying Mold

Many people struggle to identify mold in its early stages. Especially in apartments and studio housing, residents often notice dark patches on walls, ceilings, or bathrooms but are unsure whether these are mold, dirt, or just discoloration. Without accurate identification, the mold continues to spread, causing damage to the home and potential health issues such as allergies or respiratory irritation. There are currently not a lot of widely known simple and accessible tools for ordinary users to detect mold accurately using only their smartphone.

2) Limited Access to Reliable Guidance

Even when mold is detected, many people are unsure how to handle it correctly, whether to clean it themselves, what cleaning agents to use, or when to seek professional intervention. Searching online often gives inconsistent or unreliable advice. This causes repeated mold problems and unnecessary expenses for cleaning services or repairs.

3) Need for a Simple and Interactive Solution

Current mold prevention solutions are often hardware-centric and expensive, such as dehumidifiers or sensors, and a reactive approach, which is cleaning after mold appears. There is a gap in the market for a software-only solution that supports users in all stages: photo-based mold identification, and guidance via chatbot. A tool with AI-driven systems that combine all these features would be more affordable, easier to use, and could help reduce mold-related damage and health issues across many households.

C. Research on Related Applications

1) Mould Identifier: AI Scanner

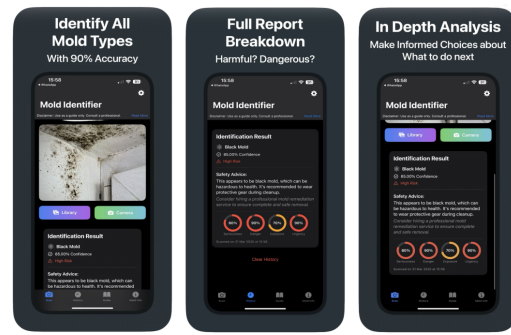


Fig. 1. Mould Identifier: AI Scanner

Mould Identifier is an AI-powered mobile application designed to facilitate easy detection and analysis of mold. By capturing or uploading a photo, the app identifies the mold type, confidence score, assesses associated risks, and provides safety guidance, including a percentage-based indication of severity, danger, exposure, and urgency. The application also features additional informational pages that offer detailed descriptions of different mold types and step-by-step usage guides for effective monitoring and prevention. A history log enables users to track past scans, while the user-friendly interface ensures effortless navigation and accessibility for casual users.

2) Mold Identifier: AI Scan

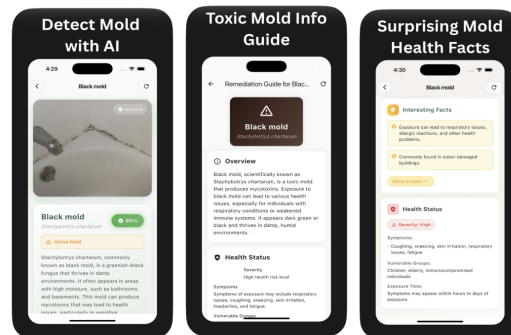


Fig. 2. Mold Identifier: AI Scan

Mold Identifier is an AI-powered application that detects mold and provides detailed information about the identified species. Upon opening the app, the camera interface launches automatically, allowing users to capture or upload an image for analysis. The AI recognizes the mold type and delivers comprehensive descriptions, including characteristics, typical habitats, and care information.

3) Mold Finder AI, Inspection

Mold Finder AI provides a solution for detecting,

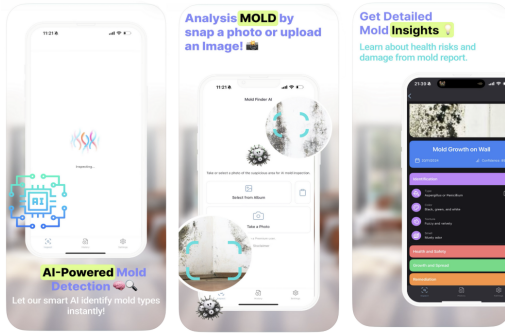


Fig. 3. Mold Finder AI, Inspection

analyzing, and understanding mold directly from a mobile device. Utilizing advanced AI technology, the app allows users to capture a photo or upload an existing image, after which it automatically identifies the mold type, analyzes the image, and specifies the location of the mold within the captured area, along with a confidence score. In addition to detection, the app delivers comprehensive analyses covering color, texture, health and safety risks, growth and spread patterns, and recommended remediation measures.

4) Virtual Mold Inspection

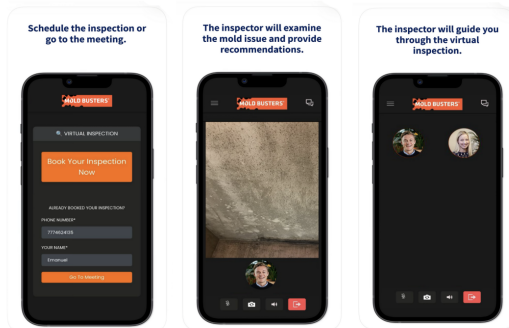


Fig. 4. Virtual Mold Inspection

Virtual Mold Inspection is an application platform that enables property owners or homeowners to schedule appointments with certified mold professionals for remote, real-time inspections. Users can select a convenient day and time, then conduct a video call through the app, during which the professional inspector guides the user in examining areas of concern, evaluates potential mold issues, and provides tailored advice on prevention and remediation. Unlike AI-based applications, this platform relies on human expertise rather than automated analysis, which may limit the speed and scalability of assistance. However, the involvement of trained professionals ensures higher accuracy in identifying mold presence and assessing

associated risks.

5) MoldAI

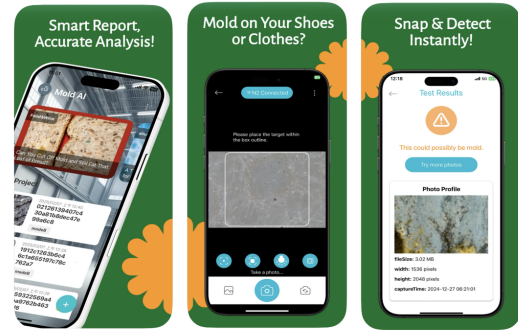


Fig. 5. MoldAI

MoldAI is an artificial intelligence–based mold detection application integrated with YCM hardware to provide real-time analysis of mold presence across various surfaces, including walls, shoes, fabrics, and industrial materials. The system incorporates a camera interface that guides users to align the target area within a designated box outline and set the shooting location to ensure consistent image capture. Upon taking or uploading an image, the AI model processes the visual data and classifies whether the area contains mold or not, delivering instant diagnostic feedback. The application also features a dashboard where users can review stored results and access a curated news section that links to external informational resources. However, the current implementation indicates that the detection functionality remains in a prototype phase, with limited real-time analytical performance.

III. REQUIREMENT ANALYSIS

1) Common Features

(a) Sign-Up and Log-In

- Enter email: Used for login and account identification.
- Enter password: Must be at least six characters long for security.
- Enter username: Used as the display name within the app. Once registered, users can log in using their email and password to access all AI-powered functionalities and save mold detection history.

2) User Specific Features

(a) Mold Photo Detection

Users can upload a photo or take a picture using their device's camera to check whether mold is present using AI image detection. The following are the information that will be displayed:

- Detection Result
- Location (entered manually by the user for better customization)
- Timestamp

(b) **Home Dashboard (My Home):**

The main screen displays the history, chatbot icon, settings icon and camera icon.

(c) **History Log:**

The application maintains a detection history archive where users can review previous AI analysis results in a chronological list. The history displays the location and the previous uploaded photos with the results.

(d) **AI Chatbot Assistant:**

The integrated chatbot allows users to ask more information on mold-related questions, such as “What specific products should I use to clean this?”. It leverages a simple NLP model to generate contextual, easy-to-understand advice and explanations.

(e) **Settings:**

The Settings section includes User Profile and About GomMold. In the User Profile section, users can customize their username and change their password. In the About GomMold section, information about the application is provided.

IV. SOFTWARE DEVELOPMENT PLATFORM

A. Development Platform

- **MacOS**

MacOS is the chosen and primary chosen development platform as it has software engineering operations like version control management and executing Python script for AI model training are made-easier by its robust UNIX-based interface. Moreover, it is perfect for creating and merging machine learning models due to its excellent support for AI frameworks like TensorFlow and PyTorch. Also, MacOS provides a reliable platform and smooth environment to unify the software engineering and AI components for this project.

- **Windows**

Windows is a popular and versatile operating system, which makes it a great platform for this project’s development. Windows user-friendly interface simplifies environment configuration and project management, while its robust community support ensures quick and fast. Moreover, this project may use dedicated GPUs for quicker model training because of its strong hardware support. In summary, Windows offers a complete and adaptable environment for creating GomMold application’s cross-platform components.

B. Tools and Language

- **Dart**

Dart is the programming language used by Flutter, developed and maintained by Google. It is a modern, object-oriented, and type-safe language designed for

building fast, reliable, and visually appealing applications across multiple platforms such as Android, iOS, web, and desktop. Dart compiles to native code for mobile and desktop, which ensures smooth performance and quick UI rendering. In this project, Dart is used to develop the frontend mobile application, enabling interactive user experiences and seamless navigation between features such as photo upload, weather display, and AI-based mold detection results. Dart also acts as the bridge connecting the app to backend services, including Firebase for authentication and data storage, and the Flask API for AI model predictions and chatbot communication.

- **Firebase**

Firebase is a cloud-based Backend-as-a-Service (BaaS) platform developed by Google, offering essential backend tools such as authentication, cloud storage, real-time databases, and analytics. In this project, Firebase plays a central role in managing user accounts, detection history, and image storage. Firebase Authentication secures user registration and login, Cloud Firestore stores detection results and environmental data, and Firebase Storage maintains uploaded mold images. The platform’s scalability and integration with Flutter allow real-time synchronization between the user interface and backend, ensuring that users can access their analysis results and personalized recommendations instantly and securely.

- **Flask**

Flask is a lightweight and flexible Python-based micro web framework recognized for its simplicity, scalability, and adaptability in modern web development. Its minimalist architecture allows developers to integrate only essential components, promoting high customization, modularity, and efficient system design. Unlike larger frameworks such as Django, Flask offers greater control and faster performance due to its minimal abstraction layers, making it ideal for data-driven and AI-powered applications. In this project, Flask can be used to build RESTful APIs that connect the image analysis and conversational modules into a unified backend system. This implementation enables real-time data exchange, accurate risk assessment, and intelligent feedback, ensuring efficiency and scalability for future smart-home integrations.

- **Flutter**

Flutter is an open-source UI software development toolkit created by Google for building visually appealing, cross-platform applications from a single codebase. Using the Dart language, it provides a reactive programming model and a rich collection of pre-designed widgets that allow rapid interface design and customization. Flutter’s hot reload feature supports instant code changes, improving

developer productivity and testing efficiency. In this project, Flutter is used to develop the mobile application interface, allowing users to upload photos, view mold detection results, access weather-based risk analysis, and interact with the AI-powered chatbot. Its integration with Firebase and Flask API enables real-time updates and seamless data exchange.

- Javascript

JavaScript is a dynamic and event-driven programming language that enables interactive web functionality and seamless data communication between the client and server. Although it is not used directly in this project's core development, JavaScript underpins Firebase's SDKs and cloud functions, which facilitate real-time data synchronization between the mobile app and the backend. Through these SDKs, JavaScript enables secure and efficient operations for user authentication, data storage, and file management without requiring a custom backend. Its asynchronous capabilities allow the system to handle real-time updates such as saving detection results, retrieving history, or synchronizing user preferences, improving the overall responsiveness and scalability of the application.

- Latex

LaTeX acts as the main tool for preparing this project's technical documentation. It helps to format paperwork professionally and consistently by following the IEEE report structure accordingly. This project uses an online LaTeX editor like Overleaf to jointly prepare and compile the document within team members. Overleaf's cloud-based platform enables real-time writing, editing, commenting and document review without version conflicts. Plus, it allows users to do automatic table of contents creation, referencing and also mathematical equations which enhance the user's quality and readability of the report documentation. So, by using LaTeX in Overleaf, it helps the team to create and polish team project documentations and paperworks.

- OpenAI

OpenAI, founded in 2015, is a leading artificial intelligence research organization dedicated to developing safe and beneficial AI technologies. Its advanced models, such as GPT and DALL-E, provide strong capabilities in natural language understanding and generation, which can be integrated into applications through APIs. In this project, OpenAI is used to power a simple conversational chatbot that assists users by answering mold-related questions and providing general information in a friendly and accessible way.

- Python

Python is a high-level, general-purpose programming language known for its simplicity and versatility. It is

widely used for backend development, data analysis, machine learning, and automation. In this project, Python serves as the core backend and AI development language, handling the logic for image processing, machine learning, and integration with external APIs. The Flask web framework, written in Python, is used to build the backend API that connects the mobile application to the AI model. Python libraries such as PyTorch and OpenCV are employed for mold detection and image preprocessing, enabling the system to identify potential mold areas and generate predictions based on uploaded images. Additionally, Python facilitates communication with third-party services like OpenAI API for chatbot responses, ensuring that all intelligent features operate smoothly and efficiently within the backend ecosystem.

- Pytorch

PyTorch is one of the primary frameworks used in the development of this project's AI model. It is a powerful open-source deep learning library that enables efficient training and deployment of neural networks using Python. PyTorch provides an intuitive and flexible platform for building and experimenting with modern AI architectures, including the YOLO model. Its dynamic computation graph allows developers to modify model behavior on the fly, making debugging and experimentation significantly easier. PyTorch's ecosystem also integrates seamlessly with scientific computing tools and dataset workflows, supporting smooth model development, evaluation, and refinement. In this project, PyTorch plays a central role in training the mold detection model and exporting the final weight file used by the backend for real-time inference.

- Railway

Railway is a modern cloud deployment platform that allows developers to run applications without managing complex server setups. It provides a simple interface for building, hosting, and scaling applications directly from a GitHub repository, using an automated Linux-based environment. Railway handles the runtime configuration, environment variables, and deployment process, making it much easier and faster compared to traditional cloud hosting. In this project, Railway is used to deploy the Flask backend that powers user authentication, image processing, and machine learning inference. It securely stores API keys, connects seamlessly with Firebase, and provides real-time logs to troubleshoot errors efficiently. By hosting the backend on Railway, the system achieves stable performance, quicker updates, and smooth communication with both the Flutter frontend and the OpenAI chatbot.

- Roboflow

Roboflow is an essential tool used in this project

for dataset creation, management, and preparation. It provides a powerful platform for organizing images, performing annotations, and generating high-quality training datasets for computer vision tasks. Roboflow offers automated preprocessing features such as image resizing, augmentation, and format conversion, which help improve model robustness and performance. Its intuitive web interface makes it easier to handle large and diverse image collections, especially for complex tasks like mold detection where consistent labeling is critical. By exporting datasets directly into formats compatible with YOLO, PyTorch, and ONNX workflows, Roboflow streamlines the entire dataset pipeline. In this project, Roboflow played an important role in curating, augmenting, and exporting the mold dataset, ensuring the model received clean and well-structured data for effective training.

C. Software in Use

- Figma



Fig. 6. Figma

Figma is a collaborative online design and prototype tool for user interface (UI), user experience (UX), and digital products. It is one of the most popular tools in the design industry due to its on real-time collaboration. It allows multiple team members to work simultaneously. This is particularly advantageous for this project, as all team members can access and contribute to designs seamlessly. Figma enables users to see the prototype and test the app flow before any code is written. It also provides a dedicated workspace for developers to inspect design elements, extract assets, and translate the design directly into code.

- Github



Fig. 7. Github

Github is the primary code management and version control in this project. It enables team members to work on code independently without overwriting each other's work. Besides that, it allows to organize branches for different functionality, manage project repositories and as well as making sure all the code changes are recorded with commits. It also prevents data loss in the event of accidental deletion of local files by acting as a secure online backup. Vs code also integrates well with GitHub by offering effortless synchronization between code editing and version control. This project utilizes Github to ensure effective collaboration, accountability and transparency across all phases of the project's development's.

- Google Colab



Fig. 8. Google Colab

Google Colab serves as the primary model training environment for this project. It provides a cloud-based platform equipped with free GPU and TPU resources, enabling efficient training of the YOLO model without requiring high-performance hardware from team members. Colab offers a collaborative notebook interface that allows code, results, and experiments to be clearly documented and shared. This ensures that training workflows remain reproducible, organized, and easy to update as the dataset evolves. The integration with Google Drive also simplifies data management by enabling seamless loading, saving, and exporting of model weights. Through its accessibility, shared workspace features, and GPU-accelerated environment, Google Colab supports fast experimentation and consistent model development.

- Notion



Fig. 9. Notion

Notion functions as the primary project management and collaboration platform for the team. It facilitates

the organization’s task list,schedules, meeting notes and essential resources within a singular platform. Notion is used to allocate assignments, track the progress and make sure each team member is fully aware of their duties and timelines. Plus, it enables real-time editing and offers simultaneous visibility of updates made by one user to others. By this transparency, it minimizes communication gaps and helps to smoothen the coordination between frontend, backend and AI developer as well. Notion is essential for keeping team alignment and productivity during the development process.

• Postman



Fig. 10. Postman

Postman is an open-source API development and testing tool that allows developers to send requests, analyze responses, and troubleshoot API efficiently. It supports various request types and includes features such as organizing endpoints and automated testing. In this project, Postman is used as the central platform for validating and testing all API interactions before they are integrated into the system. By using Postman as an API testing environment, this project ensures the backend functions correctly.

• Visual Studio Code



Fig. 11. Visual Studio Code

Visual Studio Code (VS Code) is a lightweight, open-source code editor developed by Microsoft. It supports multiple programming languages and includes features such as syntax highlighting, debugging, and a wide range of plugin extensions. This project uses VS Code as the main platform for all stages of code development. It serves as the central hub where both front-end and back-end code is written, tested, and managed before being deployed to the physical application. By using VS Code as the primary development environment, this project can

seamlessly integrate all components, ensuring that the final project is well-organized and easily maintainable.

D. Team’s Development Environment

Name	Environment
Aina Nur Insyeerah	Macbook Air M4, 24GB MacOS Tahoe 26.0.1 Python 3.13.7
Nur Ain Syafiqah	Macbook Air M3, 24GB MacOS Sequoia 15.6.1 Python 3.9.6
Nur Shaqeerah	Macbook Air M1, 8GB MacOS Sequoia 15.6.1 Python 3.13.7
Yumni Karmila	HP Pavillion 15, 12th Gen Intel(R) Core(TM) i7-1255U (1.70 GHz), 512GB, 16 RAM, 2GB GPU, Python 3.10.18

E. Cost Estimation

This project incurs no additional financial cost, as it relies entirely on open-source and free software. Development is carried out using personal laptops, eliminating hardware expenses. Additionally, the project takes advantage of cloud services, such as computing and storage resources, which provide scalable and flexible infrastructure without extra cost.

F. Task Distribution

Tasks	Name	Descriptions
Frontend Developer	Nur Ain Syafiqah, Yumni Karmila	Frontend developers build the user interface, manage interactivity, and integrate real-time data using Flutter and Dart. The first developer, the UI implementation Specialist transforms Figma designs into responsive mobile interfaces, handling navigation, animations, and Firebase authentication with focus on usability and performance. The second developer is the Data Integration and Visualization Specialist who connects the Flask backend with the frontend through APIs for mold detection, manages data visualizations, synchronization and collaborates with backend and AI teams for seamless functionality.
UI-UX Designer	Yumni Karmila	The UX Designer shapes the overall user experience and visual design of the app, ensuring accessibility, usability,and consistency. Using Figma, the designer create interface layouts, color schemes, typography, and user flows through wireframes and interactive prototypes. Usability testing is conducted to refine designs based on feedback and collaborate with frontend developers to ensure accurate visual implementation. The

		designer also maintains a design system for consistent branding.
AI Developer	Nur Shaqeerah	The AI Developer is responsible for deploying a YOLO-based machine learning model to detect and classify mold in real time. This includes collecting and preprocessing image data, training and fine-tuning the YOLO model in Google Colab using PyTorch for higher accuracy, and integrating the trained model into the app through Flask API endpoints for seamless communication between the AI, backend, and frontend. By leveraging YOLO's real-time detection capabilities, this role ensures automated, scalable, and reliable mold recognition.
Backend Developer	Aina Nur Insyeerah	The Backend Developer designs the database, builds APIs, and ensures smooth data flow between the frontend, backend, and AI components. Using Python and Flask, the developer creates a robust server system that processes user data and returns real-time responses. The role integrates external APIs, uses an ONNX-based cloud model for mold detection, implements the OpenAI API for chatbot features, and manages secure data storage with Firebase. The backend is deployed on Railway to ensure stable, scalable, and efficient cloud performance.

V. SPECIFICATIONS

A. Initial Page (Splash Screen)



Fig. 12. Initial page

The initial screen appears for a few seconds upon launching the application, serving as a splash page that introduces the GomMold brand and ensures all required system resources are initialized (e.g., Firebase, API connections).

- **Process**
 - The app loads core dependencies (Flask API endpoints, AI model, Firebase configurations).
 - Displays GomMold logo and animation for 3–5 seconds.
 - Automatically transitions to the Log-In Page once initialization completes.
- **System Response**
 - Success: Redirects to Log-In Page.
 - Failure: Displays “Connection Error” message if back-end initialization fails.

B. Log In

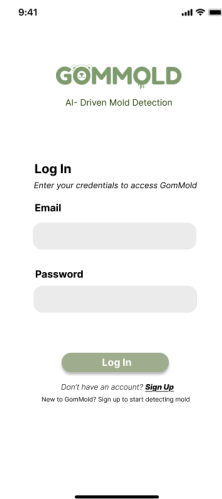


Fig. 13. Log In page

This page allows registered users to access their account using a valid email and password.

- **Input Fields:**
 - Email: Used for account identification (case-insensitive).
 - Password: Must match with the securely hashed password stored in the database.
- **Buttons:**
 - Log In: Validates Credentials
 - Sign Up: Redirects to Sign Up page for new users.
- **Process Flow:**
 - 1) User enters login credentials
 - 2) Upon pressing “Log In,” the frontend sends a POST `/api/auth/login` request to the Flask backend.
 - 3) The backend checks if the email exists in the users collection in Firestore.
 - 4) If found, it validates the password using `check_password_hash`.
 - 5) On successful authentication, a JWT token (valid for 24 hours) is generated and returned to the app.
 - 6) User is redirected to the Homepage.
- **System Response**

- Success: Redirects to Homepage.
- Failure: Displays "Login failed. Try again."

C. Sign Up

Fig. 14. Sign Up page

This page allows new users to register an account.

- Input Fields:
 - Email: Must be unique in the database
 - Username: Must be unique in the database.
 - Password: Minimum of 6 characters.
- Buttons:
 - Sign Up: Registers new user.
 - Back to Log In: Returns to Log-In Page.
- Process Flow:
 - 1) User enters email, username, and password.
 - 2) Frontend sends a POST `/api/auth/signup` request to Flask backend.
 - 3) Backend checks if email or username already exist in the users collection.
 - 4) If unique, the password is hashed using Werkzeug's `generate_password_hash()` and stored securely.
 - 5) Upon successful registration, a JWT token is generated and returned.
 - 6) The user is redirected to the Log-In Page.
- System Response
 - Success: Displays "Registration successful. Please log in."
 - Failure: Displays one of the following:
 - * "Password must be at least 6 characters."
 - * "Sign up failed. Try again."

D. Homepage (Main Dashboard)

The Homepage serves as the user's control center. It displays detection history and provides quick navigation to camera, chatbot, and settings.

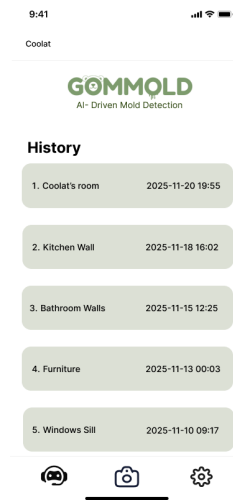


Fig. 15. Homepage

- Display Elements:
 - Middle Section (History): Displays list of past detections:
 - * Location
 - * Results (Mold / No Mold)
- Buttons Navigation Bar:
 - Settings Button (Left) – Opens Settings Page.
 - Camera Button (Center) – Opens Image Page.
 - Chatbot Button (Right) – Opens Chatbot Page.
- Process Flow:
 - 1) When the page loads, it sends GET `/api/mold/history` to the backend.
 - 2) Backend retrieves the user's history from the detections table and sends it to the frontend.
 - 3) Backend checks if username exists in the users table.
 - 4) Data is displayed in chronological order.
- System Response
 - Success: Displays recent detection history.
 - Displays "No detection history found."

E. Image Page

This page allows users to choose between capturing a photo or uploading one from their gallery for AI-based mold detection.

- Buttons:
 - Take Photo: Opens real-time camera interface.
 - Choose from Gallery: Opens device's photo picker.
 - Back: Returns to Homepage.
- Process Flow:
 - 1) When "Take Photo" is selected, app requests camera permission.
 - 2) When "Choose from Gallery" is selected, app opens local gallery.
 - 3) Once an image is chosen or captured, it navigates to the Identify Page.

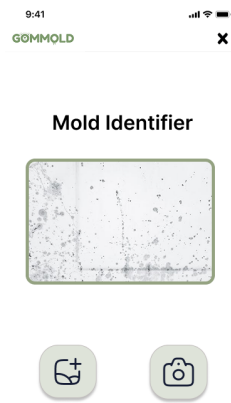


Fig. 16. Image page

- *System Response*

- Success: Redirects to Identify Page with selected image.
- Failure: Displays “Access to camera denied” if camera access is blocked.

F. Identify Page (Mold Detection Page)

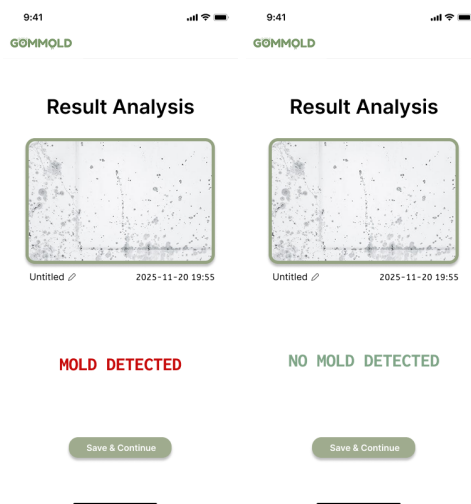


Fig. 17. Mold Detection Page

The Identify Page allows users to capture or upload an image and run mold detection using the backend AI model. Results are displayed immediately with a color-coded status to indicate safety.

- *Buttons:*

- Live Camera (real-time capture)
- Shoot Button
- Back Button (returns to Image Page)
- Result Display Area

- *Process Flow:*

- 1) User takes or selects an image from the device.
- 2) The image is sent to the backend via `POST /api/mold/detect` along with the user's authentication token.
- 3) Flask receives the uploaded file, stores it in Firebase Storage, and generates a public image URL.
- 4) The backend temporarily saves the file and passes it into the ONNX model using `predict_image()`.
- 5) The model outputs bounding boxes and confidence scores for potential mold.
- 6) Based on the filtered detections, the backend determines:
 - Red Warning – “Mold Detected” (if at least one prediction is above threshold)
 - Green Safe – “No Mold Detected” (if no valid detections are found)
- 7) The backend saves the result, predictions, image URL, and timestamp into Firestore under the `detections` collection.
- 8) The JSON response is returned to the app, containing the result, prediction details, and timestamp.
- 9) The app displays the detection result instantly on the same page.

- *System Response*

- Success: Detection result is displayed with prediction data.
- Failure: Shows “Image processing error.”

G. Chatbot Page

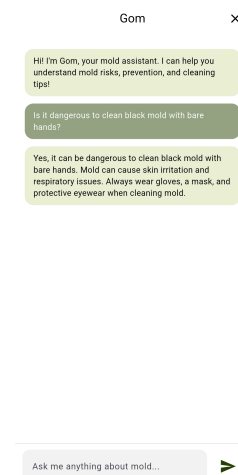


Fig. 18. Chatbot page

The chatbot allows users to interact with an AI assistant for mold-related questions and general information.

- *Components:*

- 1) Initial AI message introducing itself.
- 2) Text input field and Send button.

3) “Back” button (returns to Homepage).

- Process Flow

- 1) Upon entering, the chatbot sends a greeting via back-end API: GET /api/chatbot/start
- 2) User types a question into the chat input.
- 3) The query is sent to the backend: POST /api/chatbot/query
- 4) Backend forwards the message to the OpenAI API for language processing.
- 5) The AI generates a response and returns it to the app.

- System Response

- Success: AI reply is displayed in a chat-style conversation.
- Failure: Displays “Chat service unavailable.”

System Response

- Success: AI reply displayed in conversation format.
- Failure: Displays “Chat service unavailable.”

H. Settings Page

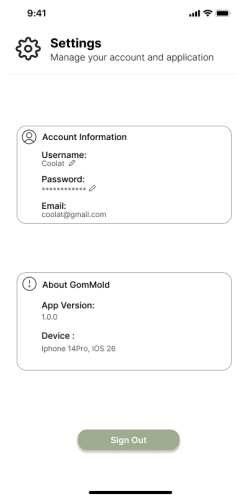


Fig. 19. Settings page

This page allows users to manage their profile information and view app details.

- Display Elements:

- Change Username Button
- Change Password Button
- About GomMold Section (App details, version info, developer credits)
- Back Button: Returns to Homepage

- Process Flow

- 1) When user clicks “Change Username” or “Change Password,” a form opens.
- 2) On submission, app sends a PATCH /api/users/update request.
- 3) Backend updates user data in the users table.
- 4) Updated info is synced with Firebase Authentication if applicable.

- System Response

- Success: Displays “Profile updated successfully.”
- Displays “Error updating profile.”

VI. ARCHITECTURE PAGE

A. Overall Architecture

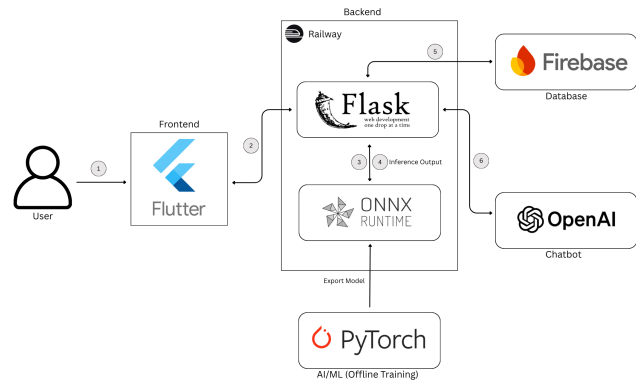


Fig. 20. Application Architecture

The GomMold system architecture is organized into two primary modules: Frontend and Backend. The Frontend module, developed using Flutter, serves as the main interface through which users interact with the application. It provides functionalities such as image upload, mold analysis viewing, detection history, and chatbot communication. The user interface emphasizes accessibility and responsiveness, allowing users to capture or select images, review results, and manage their profile settings. The frontend collects all user inputs and communicates with the backend through secure RESTful APIs, relying entirely on the backend to perform authentication, verification, and session handling.

The Backend module, implemented with the Flask framework and deployed on a Linux-based server via Railway, acts as the central processing engine of the system. It manages all authentication processes, including user registration, login, token generation, and session validation, ensuring secure access across devices. The backend also handles the core image-processing pipeline: receiving uploaded images, running them through the machine learning inference engine, interpreting prediction outputs, and returning structured JSON responses back to the Flutter application. Additionally, the backend integrates the OpenAI API to power an interactive chatbot, enabling users to ask mold-related questions and receive cleaning advice, risk explanations, and preventative guidance. By hosting the backend on Railway, GomMold benefits from faster deployment cycles, simpler configuration, and more transparent error handling.

The data storage layer is responsible for managing all user-generated and application-related data. Firebase Firestore is used to store structured data such as user profiles and detection metadata, while Firebase Storage handles unstructured data such as uploaded images and model inference results. This design ensures efficient data retrieval, real-time synchronization, and reliable image versioning across the application layers.

The machine learning component forms the core analytical engine of GomMold. The mold detection model is built using PyTorch and trained offline using datasets prepared and annotated through Roboflow. Training is conducted in Google Colab to optimize the model under various real-world lighting and surface conditions. After training, the model is exported to ONNX format to ensure compatibility with the cloud backend. Within the Flask backend, the ONNX model is loaded and executed using ONNX Runtime, allowing fast and lightweight inference for every uploaded image. The system evaluates the model's output to determine the presence of mold and its confidence level, returning the result directly to the user through the mobile application. In addition to detection, GomMold incorporates an optional chatbot powered by the OpenAI API, enabling users to obtain further clarification, cleaning tips, or preventive advice.

Collectively, these modules create an integrated ecosystem that combines real-time image analysis, cloud-based data management, and AI-assisted feedback to deliver a practical, reliable, and user-friendly solution for mold detection and prevention. The modular architecture ensures scalability, maintainability, and interoperability, positioning GomMold as a viable tool for both everyday users and future advancements in environmental health monitoring.

B. Directory Organization

TABLE I
DIRECTORY-ORGANIZATION-FRONTEND-1

Directory	File Name
Root	.flutter-plugins-dependencies .gitignore .metadata README.md analysis_options.yaml devtools_options.yaml firebase.json gomMold_frontend.iml package-lock.json package.json pubspec.lock pubspec.yaml

TABLE II
DIRECTORY-ORGANIZATION-FRONTEND-2

.dart_tool	package_config.json package_graph.json version
.dart_tool/dartpad	web_plugin_registrant.dart
.idea	modules.xml workspace.xml
.idea/libraries	Dart_SDK.xml KotlinJavaRuntime.xml
.idea/runConfigurations	main_dart.xml
android	.gitignore build.gradle.kts gommold_frontend_android.iml gradle.properties gradlew gradlew.bat local.properties settings.gradle.kts
android/app	build.gradle.kts google-services.json
android/app/src/debug	AndroidManifest.xml
android/app/src/main	AndroidManifest.xml
android/app/src/debug	AndroidManifest.xml
android/app/src/main/kotlin/ com/example/ gommold_frontend	MainActivity.kt
android/app/src/main/res /drawable-v21	launch_background.xml
android/app/src/main/res /drawable	launch_background.xml
android/app/src/main/res /mipmap-hdpi	ic_launcher.png
android/app/src/main/res /mipmap-mdpi	ic_launcher.png
android/app/src/main/res /mipmap-xhdpi	ic_launcher.png
android/app/src/main/res /mipmap-xxhdpi	ic_launcher.png
android/app/src/main/res /mipmap-xxxhdpi	ic_launcher.png

TABLE III
DIRECTORY-ORGANIZATION-FRONTEND-3

android/app/src/main/res/values-night	styles.xml
android/app/src/main/res/values	styles.xml
android/app/src/profile	AndroidManifest.xml
android/gradle/wrapper	gradle-wrapper.properties
assets/images	ChatBot.png Homepage-time.png Identify Page-Success Result.png Image Page.png LogIn Page.png Settings.png Sign Up Page.png logo.png
dist	socket.js
ios	.gitignore Podfile
ios/Flutter	AppFrameworkInfo.plist Debug.xconfig Generated.xconfig Release.xconfig flutter_export_environment.sh
ios/Runner.xcodeproj	project.pbxproj
ios/Runner.xcodeproj/project.xcworkspace	contents.xcworkspacedata
ios/Runner.xcodeproj/project.xcworkspace/xcsourcedata	IDEWorkspaceChecks.list
ios/Runner.xcodeproj/xcsourcedata/xcschemes	Runner.xscheme
ios/Runner.xcworkspace	contents.xcworkspacedata
ios/Runner.xcworkspace/xcsourcedata	IDEWorkspaceChecks.list WorkspaceSettings.xcsettings
ios/Runner	AppDelegate.swift GeneratedPluginRegistrant.h GeneratedPluginRegistrant.m GoogleService-Info.plist Info.plist Runner-Bridging-Header.h
ios/Runner/Assets.xcassets/AppIcon.appiconset	Contents.json Icon-App-1024x1024@1x.png Icon-App-20x20@1x.png Icon-App-20x20@2x.png Icon-App-20x20@3x.png Icon-App-29x29@1x.png Icon-App-29x29@2x.png Icon-App-29x29@3x.png Icon-App-40x40@1x.png Icon-App-40x40@2x.png Icon-App-40x40@3x.png Icon-App-60x60@2x.png Icon-App-60x60@3x.png Icon-App-76x76@1x.png Icon-App-76x76@2x.png Icon-App-83.5x83.5@2x.png

TABLE IV
DIRECTORY-ORGANIZATION-FRONTEND-4

ios/Runner/Assets.xcassets/LaunchImage	Contents.json
ios/Runner/Assets.xcassets/LaunchImage	Contents.json
imageset	LaunchImage.png LaunchImage@2x.png LaunchImage@3x.png README.md
ios/Runner/Base.lproj	LaunchScreen.storyboard Main.storyboard
ios/RunnerTests	RunnerTests.swift
lib	main.dart
lib/src/models	detection.dart
lib/src/screens	chatboot_page.dart homepage_time.dart identify_page.dart image_page.dart initial_page.dart login_page.dart settings_page.dart sign_up_page.dart
lib/src/services	api_service.dart, auth_service.dart, firebase_service.dart
lib/src/utills	constants.dart image_utils.dart
lib/src/widgets	card_item.dart custom_button.dart custom_textfield.dart
linux	.gitignore CMakeLists.txt
linux/flutter	CMakeLists.txt generated_plugin_registrant.cc generated_plugin_registrant.h generated_plugins.cmake
linux/runner	CMakeLists.txt main.cc my_application.cc my_application.h
macos	.gitignore Podfile Podfile.lock
macos/Flutter	Flutter-Debug.xcconfig Flutter-Release.xcconfig GeneratedPluginRegistrant.swift
macos/Runner.xcodeproj	project.pbxproj
macos/Runner.xcodeproj/project.xcworkspace/xcsourcedata	IDEWorkspaceChecks.plist
macos/Runner.xcodeproj/xcsourcedata/xcschemes	Runner.xcscheme
macos/Runner.xcworkspace	contents.xcworkspacedata
macos/Runner.xcworkspace/xcsourcedata	IDEWorkspaceChecks.plist
macos/Runner	AppDelegate.swift DebugProfile.entitlements GoogleService-Info.plist Info.plist MainFlutterWindow.swift Release.entitlements

TABLE V
DIRECTORY-ORGANIZATION-FRONTEND-5

macos/Runner/Assets.xcassets/AppIcon.appiconset	Contents.json app_icon_1024.png app_icon_128.png app_icon_16.png app_icon_256.png app_icon_32.png app_icon_512.png app_icon_64.png
macos/Runner/Base.lproj	MainMenu.xib
macos/Runner/Configs	AppInfo.xcconfig Debug.xcconfig Release.xcconfig Warnings.xcconfig
macos/RunnerTests	RunnerTests.swift
node_modules	.package-lock.json
node_modules/ws	LICENSE README.md browser.js index.js package.json wrapper.mjs
node_modules/ws/lib	buffer-util.js constants.js event-target.js extension.js limiter.js permessage-deflate.js receiver.js sender.js stream.js subprotocol.js validation.js websocket-server.js websocket.js
test	widget_test.dart
web	favicon.png index.html manifest.json
web/icons	Icon-192.png Icon-512.png Icon-maskable-192.png Icon-maskable-512.png
windows	.gitignore CMakeLists.txt
windows/flutter	CMakeLists.txt Generated_plugin_registrant.cc Generated_plugin_registrant.h generated_plugin.cmake
windows/runner	CMakeLists.txt Runner.rc flutter_window.cpp flutter_window.h main.cpp resource.h resource.h runner.exe.manifest utils.cpp utils.h wiin32_window.cpp win32_window.h
windows/runner/resources	app_icon.ico

TABLE VI
DIRECTORY-ORGANIZATION-BACKEND-1

Directory	File Name
root	.gitignore Procfile README.md requirements.txt runtime.txt
src	__init__.py app.py firebase_init.py ml_model.py token_utils.py
src/routes	__init__.py auth.py chatbot.py history.py mold.py user.py
src/models	best.onnx

TABLE VII
DIRECTORY-ORGANIZATION-AI

Directory	File Name
root	.DS_Store README.md
models	README.md (download link for the trained model because github does not allow large file upload)
training	GomMold_ML.ipynb dataset_info.md
training/images	BoxF1_curve (1).png BoxPR_curve (1).png README.md confusion_matrix (1).png

C. Module 1: Frontend

1) Purpose

The Frontend Module of GomMold serves as the primary user-facing interface, allowing individuals to interact seamlessly with the mold detection system. It manages all visual elements, user inputs, and navigation flows to deliver an intuitive and responsive experience across devices. Developed using Flutter, the frontend communicates securely with the backend server to submit images for analysis, retrieve mold detection results, and handle user authentication. Its primary objective is to provide a smooth, accessible, and reliable platform that enables users to perform mold identification efficiently while maintaining consistent performance and usability.

2) Functionality

The GomMold frontend offers comprehensive features including user authentication, secure token storage, and input handling for email and password. Users can capture images using the device camera or select from the gallery, and these images are transmitted to the backend for analysis via RESTful APIs. Detection results are displayed clearly, and navigation across pages such as login, home, and identification is handled seamlessly. The module incorporates reusable widgets and components for consistent UI design, manages application state using provider and service classes, and ensures real-time interaction with the backend. Additionally, the frontend integrates a chatbot interface for guidance and supports data visualization, history viewing, and profile management, providing a fully interactive and user-friendly mold detection experience.

3) Location of source code : https://github.com/GomMold/GomMold_FrontEnd

4) Class Component

- root :
 - .flutter-plugins-dependencies : This is a file that contains metadata for Flutter plugin dependencies used by the project.
 - .gitignore : This is a file that specifies which files and directories should be ignored by Git version control.
 - .metadata : This is a file that contains project metadata generated by Flutter, including version and configuration details.
 - README.md : This is a documentation file that provides an overview of the project and instructions for developers.
 - analysis_options.yaml : This is a configuration file that defines Dart and Flutter linting rules for static code analysis.
 - devtools_options.yaml : This is a file that configures options for Dart DevTools.

- firebase.json : This is a configuration file defining Firebase settings for the project.
- gomMold_frontend.iml : This is an IntelliJ project module file containing IDE-specific configuration.
- package-lock.json : This is a file that records exact versions of Node dependencies to ensure consistent builds.
- package.json : This is a file that defines Node scripts and dependencies used in development tooling.
- pubspec.lock : This is a lock file containing the resolved versions of Flutter packages for reproducible builds.
- pubspec.yaml : This is the core Flutter configuration file that manages dependencies, assets, and project metadata.
- .dart_tool :
 - package_config.json : This is a Dart configuration file mapping packages to filesystem locations.
 - package_graph.json : This is a file that describes dependency graph metadata for Dart packages.
 - version : This is a file that stores package version information used internally by Dart.
- .dart_tool/dartpad
 - web_plugin_registrant.dart : This is a file that registers Flutter web plugins for browser builds.
- idea
 - modules.xml : This is a file defining project modules for IntelliJ or Android Studio.
 - workspace.xml : This is a file containing IDE workspace settings, such as editor state and local preferences.
- .idea/libraries
 - Dart_SDK.xml : This is a configuration file defining the Dart SDK location for the IDE.
 - KotlinJavaRuntime.xml : This is a file specifying Kotlin and Java runtime libraries for the IDE.
- .idea/runConfigurations
 - main_dart.xml : This is a run configuration file that stores IDE execution settings for running main.dart.
- android
 - gitignore (android) : This is a file that specifies which Android-specific files and folders should be ignored by Git.
 - build.gradle.kts : This is the Gradle build configuration script for the Android module written in Kotlin DSL.
 - gomMold_frontend_android.iml : This is an Android Studio module file specific to the Android project configuration.
 - gradle.properties : This is a file that defines Gradle configuration properties such as JVM settings and build options.

- gradlew : This is the Gradle wrapper script used to execute Gradle commands on Unix-based systems.
- gradlew.bat : This is the Gradle wrapper script used to execute Gradle commands on Windows systems.
- local.properties : This is a file that specifies the Android SDK path and other local machine configurations.
- settings.gradle.kts : This is the Gradle settings file that defines the modules included in the Android project.
- android/app
 - build.gradle.kts : This is the Gradle build configuration script for the Android module, responsible for defining dependencies, build types, and project settings.
 - google-services.json : This is a Firebase configuration file that connects the Android app to Firebase services such as authentication and cloud storage.
- android/app/src/debug
 - AndroidManifest.xml : This is a manifest file used specifically for the debug build configuration.
- android/app/src/main
 - AndroidManifest.xml : This is the main manifest file that defines Android app metadata, permissions, and activity declarations.
- android/app/src/main/kotlin/com/example/gommold_frontend
 - MainActivity.kt : This is the main Android activity file that initializes Flutter and bridges the Flutter engine with the native Android environment.
- android/app/src/main/res/drawable-v21
 - launch_background.xml : This is an Android drawable file for configuring the launch screen background on devices running API level 21 and above.
- android/app/src/main/res/drawable-v21
 - launch_background.xml : This is an Android drawable file for configuring the launch screen background on devices running API level 21 and above.
- android/app/src/main/res/mipmap-mdpi
 - ic_launcher.png : This is the medium-density launcher icon for Android devices (MDPI).
- android/app/src/main/res/mipmap-xhdpi
 - ic_launcher.png : This is the extra-high-density launcher icon (XHDPI).
- android/app/src/main/res/mipmap-xxhdpi
 - ic_launcher.png : This is the extra-extra-high-density launcher icon (XXHDPI).
- android/app/src/main/res/mipmap-xxxhdpi
 - ic_launcher.png : This is the extra-extra-extra-high-density launcher icon (XXXHDPI).
- android/app/src/main/res/values-night
 - styles.xml : This file contains theme styles for dark mode (night) UI.
- android/app/src/main/res/values
 - styles.xml : This file contains default UI theme styles for Android light mode.
- android/app/src/profile
 - AndroidManifest.xml : Manifest file used by the profile build configuration for profiling.
- android/gradle/wrapper
 - gradle-wrapper.properties : Defines the Gradle wrapper configuration.
- assets/images
 - ChatBot.png : This is an image asset used for the chatbot interface.
 - Homepage-time.png : This is an image representing the homepage or detection history UI.
 - Identify Page – Success Result.png : This is an image used for displaying a successful detection outcome.
 - Image Page.png : This is an image asset for the image analysis page UI.
 - Login Page.png : This is a login screen asset representing authentication UI.
 - Settings.png : This is an image used for the settings interface.
 - Sign Up Page.png : This is an image used for the registration screen.
 - logo.png : This is the main application logo for branding purposes.
- dist
 - socket.js : Manages socket or real-time connection logic.
- ios
 - .gitignore : This is a file specifying which iOS files and directories should be excluded from Git version control.
 - Podfile : This is a CocoaPods configuration file used to manage iOS-specific dependencies for the Flutter application.
- ios/Flutter
 - AppFrameworkInfo.plist : This is a configuration file for Flutter's iOS framework, defining meta-data required by the iOS system.
 - Debug.xcconfig : This is an Xcode configuration file that defines settings for the debug build of the iOS Flutter module.
 - Generated.xcconfig : This is a configuration file generated by Flutter that includes build settings and paths needed for the iOS build system.
 - Release.xcconfig : This is an Xcode configuration

- file that specifies build settings for the release version of the iOS app.
- flutter_export_environment.sh: This is a shell script generated by Flutter that exports environment variables used during the iOS build process.
- ios/Runner.xcodeproj
 - project.pbxproj: This is the main project configuration file for Xcode, containing information about build targets, file references, and project settings.
- ios/Runner.xcodeproj/project.xcworkspace
 - contents.xcworkspacedata: This is a workspace definition file that describes how Xcode organizes and loads the project workspace.
- ios/Runner.xcodeproj/project.xcworkspace/xcsourcedata
 - IDEWorkspaceChecks.list: This is a file generated by Xcode to store workspace validation checks and compatibility warnings.
- ios/Runner.xcodeproj/xcsourcedata/xcschemes
 - Runner.xscheme : This is an Xcode scheme file that defines how the app is built, run, profiled, and tested.
- ios/Runner.xcworkspace
 - contents.xcworkspacedata: This is a file that defines the workspace layout and how Xcode groups related projects.
- ios/Runner.xcworkspace/xcsourcedata
 - IDEWorkspaceChecks.list : This is a workspace-level validation file storing compatibility and version checks for Xcode
 - WorkspaceSettings.xcsettings : This is a configuration file storing workspace-level user interface and environment settings.
- ios/Runner
 - AppDelegate.swift : This is the main iOS application entry file responsible for initializing the Flutter engine and handling app lifecycle events.
 - GeneratedPluginRegistrant.h : This is a header file automatically generated to register Flutter plugins for use in the iOS project.
 - GeneratedPluginRegistrant.m : This is the implementation file that executes plugin registration when the app starts.
 - GoogleService-Info.plist : This is the Firebase configuration file containing API keys and identifiers required for Firebase services on iOS.
 - Info.plist : This is the metadata configuration file for iOS containing permissions, bundle identifiers, and application properties.
 - Runner-Bridging-Header.h : This is a bridging header that allows Swift and Objective-C code to interoperate in the iOS project.
- ios/Runner/Assets.xcassets/AppIcon.appiconset
 - Contents.json : This is a metadata file describing the icon assets included in the app icon set.
 - Icon-App-1024x1024@1x.png : This is the 1024x1024 icon used for App Store listing and high-resolution branding.
 - Icon-App-20x20@1x.png : This is an app icon for small iOS interface elements at 1x resolution.
 - Icon-App-20x20@2x.png : This is a scaled 2x version of the 20x20 icon used for higher-resolution displays.
 - Icon-App-20x20@3x.png : This is a 3x scaled version of the 20x20 icon for high-density screens.
 - Icon-App-29x29@1x.png : This is the 29x29 icon used for iOS settings and spotlight searches.
 - Icon-App-29x29@2x.png : This is the 2x resolution version of the 29x29 icon.
 - Icon-App-29x29@3x.png : This is the 3x resolution version of the 29x29 icon.
 - Icon-App-40x40@1x.png : This is a 40x40 icon used in various iOS UI contexts.
 - Icon-App-40x40@2x.png : This is a 2x scaled version of the 40x40 icon.
 - Icon-App-40x40@3x.png : This is a 3x scaled version of the 40x40 icon.
 - Icon-App-60x60@2x.png : This is a 2x scaled version of the 60x60 main app icon.
 - Icon-App-60x60@3x.png : This is a 3x scaled version of the 60x60 icon used for app home screen.
 - Icon-App-76x76@1x.png : This is a 76x76 iPad home screen icon.
 - Icon-App-76x76@2x.png : This is a 2x version of the 76x76 icon for retina iPads.
 - Icon-App-83.5x83.5@2x.png : This is a scaled icon used for iPad Pro devices.
- ios/Runner/Assets.xcassets/LaunchImage.imageset
 - Contents.json : This is a metadata file that defines the launch images used for the iOS splash screen.
 - LaunchImage.png : This is the default launch image displayed when the app starts.
 - LaunchImage@2x.png : This is the higher-resolution version of the launch image for 2x density devices.
 - LaunchImage@3x.png : This is the 3x resolution launch screen image for high-density iOS devices.
- ios/Runner/Base.lproj
 - README.md : This is a documentation file included within the iOS project's localization folder, typically containing notes or configuration instructions.
 - LaunchScreen.storyboard : This is a storyboard file that defines the visual layout for the app's launch screen on iOS.
 - Main.storyboard : This is a storyboard file that defines interface components for native iOS screens

when needed by the Flutter integration.

- ios/RunnerTests
 - RunnerTests.swift: This is a Swift test file used for writing unit tests for the iOS portion of the Flutter application.
- lib
 - main.dart : This is the primary entry point of the Flutter application that initializes routing and assembles the overall app structure.
- lib/src/models
 - detection.dart : This is a file that defines the data model for mold detection, including fields such as prediction results, confidence scores, and image metadata.
- lib/src/screens
 - chatbot_page.dart : This is a file that displays the chatbot user interface and handles real-time AI interactions.
 - homepage_time.dart: This is a file that displays detection history, arranged chronologically.
 - identify_page.dart : This is a file that shows mold detection results returned by the backend or ML model.
 - image_page.dart : This is a file for selecting, previewing, and uploading images for detection.
 - initial_page.dart: This is the first screen loaded when the app starts, typically a splash or welcome page.
 - login_page.dart : This is a file that manages user login with form validation and Firebase authentication.
 - settings_page.dart : This is a file that manages user settings such as preferences and profile adjustments.
 - sign_up_page.dart : This is a file for user registration, collecting required information for account creation.
- lib/src/services
 - api_service.dart : This is a file that manages communication with the backend API, including HTTP requests.
 - auth_service.dart : This is a file responsible for handling user authentication, including login, logout, and token management.
 - firebase_service.dart : This is a file for handling Firebase operations such as image uploads and cloud storage access.
- lib/src/utills
 - constants.dart : This is a file that stores global constants including theme colors, API endpoints, and reusable configuration values.
 - image_utils.dart : This is a file containing image-related helper functions such as formatting, resizing, and path handling.
- lib/src/widgets
 - card_item.dart : This is a reusable UI component that displays content inside a styled card.
 - custom_button.dart : This is a custom reusable button component used across multiple screens.
 - custom_textfield.dart : This is a reusable text field widget for form input and user data entry.
- linux
 - .gitignore : This is a file specifying which Linux-related files should be ignored by Git version control.
 - CMakeLists.txt : This is a configuration file for building the Flutter Linux desktop application using CMake.
- linux/flutter
 - CMakeLists.txt : This is a CMake configuration file specific to Flutter's Linux plugin system.
 - generated_plugin_registrant.cc : This is a file that registers Flutter plugins for the Linux build.
 - generated_plugin_registrant.h : This is the header file containing declarations for plugin registration.
 - generated_plugins.cmake: This is a CMake script automatically generated to include necessary plugins during the build.
- linux/runner
 - ClaceLists.txt : This is a file defining build configurations for the Linux runner (desktop executable).
 - main.cc : This is the main C++ file that starts the Flutter engine for Linux.
 - my_application.cc : This is a file defining application-level behavior for the Linux desktop application.
 - my_application.h : This is the header file associated with my_application.cc, defining class structures and declarations.
- macos
 - .gitignore : This is a file specifying ignored files for the macOS portion of the Flutter project.
 - Podfile : This is a configuration file for managing macOS CocoaPods dependencies.
 - Podfile.lock: This is a lockfile that stores resolved versions of CocoaPods dependencies for macOS.
- macos/Flutter
 - Flutter-Debug.xcconfig : This is a configuration file specifying build settings for the macOS debug version of the Flutter module.
- macos/Runner.xcodeproj
 - project.pbxproj : This is the core Xcode project configuration file that defines build targets, file references, frameworks, and build settings for the macOS application.
- macos/Runner.xcodeproj/project.xcworkspace/xcsourcedata

- IDEWorkspaceChecks.plist : This is a file generated by Xcode that records workspace validation checks and compatibility warnings.
- macos/Runner.xcodeproj/xcschemes
 - Runner.xcscheme : This is an Xcode scheme file that defines how the macOS application is built, run, profiled, and debugged.
- macos/Runner.xcworkspace
 - contents.xcworkspacedata : This is a workspace file that describes how the macOS Flutter project and related components are grouped in Xcode.
- macos/Runner.xcworkspace/xcschemes
 - IDEWorkspaceChecks.plist : This is a workspace-level validation and compatibility check file generated by Xcode.
- macos/Runner
 - AppDelegate.swift : This is the application entry point for the macOS version, responsible for initializing the Flutter engine.
 - DebugProfile.entitlements : This is a file specifying macOS entitlements (permissions) for the debug configuration.
 - GoogleService-Info.plist : This is a configuration file for integrating Firebase services into the macOS version of the app.
 - Info.plist : This is the metadata configuration file for the macOS application, defining identifiers, permissions, and app properties.
 - MainFlutterWindow.swift : This is a file defining the main application window for macOS, including window behavior and Flutter view setup.
 - Release.entitlements : This is an entitlements file specifying permissions and capabilities for the release version of the macOS application.
- macos/Runner/Assets.xcassets/AppIcon.appiconset
 - Contents.json : This is the asset metadata file describing each icon used by the macOS application.
 - app_icon_1024.png : This is a 1024×1024 macOS app icon used for high-resolution displays and App Store listings.
 - app_icon_128.png : This is a 128×128 icon used in various macOS UI contexts.
 - app_icon_16.png : This is a 16×16 icon used for finder previews or small interface elements.
 - app_icon_256.png : This is a 256×256 icon used for larger interface elements.
 - app_icon_32.png : This is a 32×32 macOS icon used for settings or system dialogs.
 - app_icon_512.png : This is a 512×512 icon for high-resolution macOS interface displays.
 - app_icon_64.png : This is a 64×64 icon displayed in macOS applications and menus.
- macos/Runner/Base.lproj
 - MainMenu.xib : This is an Interface Builder file that defines the macOS menu bar and application menu structure.
- macos/Runner/Configs
 - AppInfo.xcconfig : This is a configuration file defining core application-level build settings for macOS.
 - Debug.xcconfig : This is a file containing build settings for the debug version of the macOS application.
 - Release.xcconfig : This is a file specifying build settings optimized for release builds.
 - Warnings.xcconfig : This is a file containing compiler warning settings and build-level diagnostic configurations.
- macos/RunnerTests
 - RunnerTests.swift : This is a Swift file used for implementing unit tests for the macOS version of the Flutter application.
- node_modules
 - package-lock.json : This is an autogenerated lock file used by Node.js to track exact versions of installed dependencies for consistency.
- node_modules/ws
 - LICENSE : This is the license file for the WebSocket (ws) package.
 - README.md : This is a documentation file describing usage and features of the ws library.
 - browser.js : This is a file that provides browser-compatible WebSocket utilities.
 - index.js : This is the main entry file for the WebSocket implementation.
 - package.json : This is a metadata file describing the ws package, version, and dependencies.
 - wrapper.mjs : This is an ES module wrapper for the WebSocket implementation.
- node_modules/ws/lib
 - buffer-util.js : This is a JavaScript utility file that provides optimized buffer operations for WebSocket data handling.
- node_modules/ws (Additional Library Files)
 - constants.js : This file defines constant values used throughout the WebSocket implementation, such as protocol codes and configuration limits.
 - event-target.js : This is a file that implements an EventTarget-like API, allowing WebSocket instances to handle events in a browser-compatible manner.
 - extension.js : This file manages WebSocket protocol extensions, such as permessage-deflate.
 - limiter.js : This module provides rate-limiting utilities to prevent excessive WebSocket message processing.
 - permessage-deflate.js : This file implements the permessage-deflate compression extension for

- WebSocket data frames.
- receiver.js : This file is responsible for receiving, parsing, and validating incoming WebSocket frames.
- sender.js : This module manages the sending of WebSocket frames from the server or client.
- stream.js : This provides streaming utilities used for WebSocket data handling.
- subprotocol.js : This file manages WebSocket subprotocol negotiation between client and server.
- validation.js : This module contains validation utilities to ensure WebSocket frames follow protocol rules.
- websocket-server.js : This file implements the WebSocket server, handling connections and broadcast logic.
- websocket.js : This is the main WebSocket implementation file that provides the core WebSocket class and protocol logic.
- test
 - widget_test.dart : This is a Dart test file used for writing widget-level unit tests for the Flutter application interface.
- web
 - favicon.png : This is the small icon displayed in the browser tab when running the Flutter web version.
 - index.html : This is the main HTML file for Flutter Web, responsible for loading Flutter’s compiled JavaScript code.
 - manifest.json : This is a web app manifest file that defines metadata for Progressive Web App (PWA) features such as app name, icons, and theme color.
- web/icons
 - Icon-192.png : This is a 192×192 web app icon used for PWA installation and app shortcuts.
 - Icon-512.png : This is the 512×512 PWA icon used for high-resolution displays.
 - Icon-maskable-192.png : This is a 192×192 maskable icon used for adaptive icons on modern devices.
 - Icon-maskable-512.png : This is a 512×512 maskable icon for high-resolution adaptive icons.
- windows
 - .gitignore : This file defines which Windows-specific files and folders should be excluded from Git version control.
 - CMakeLists.txt : This is the main CMake build configuration file used to compile the Flutter Windows desktop application
- windows/flutter
 - CMakeLists.txt : This is a CMake file containing build instructions for Flutter plugins on Windows.
 - Generated_plugin_registrant.cc : This is a C++ file that registers all Flutter plugins for the Windows build environment.
 - Generated_plugin_registrant.h : This is the header file containing declarations for plugin registration functions.
 - generated_plugins.cmake : This CMake file automatically includes all required Flutter plugins during the build process.
- windows/runner
 - CMakeLists.txt : This is a build configuration file that defines build rules for the Windows runner executable.
 - Runner.rc
 - flutter_window.cpp : This contains Windows resource definitions (icons, metadata) for the Windows application.
 - flutter_window.h : This is the header file containing declarations for Flutter window behavior.
 - main.cpp : This is the main entry point for the Windows desktop version of the Flutter application.
 - resource.h : This is a header file containing resource identifiers used by the Windows app.
 - runner.exe.manifest : This manifest file defines Windows application metadata such as DPI awareness and execution privileges.
 - utils.cpp : This file contains utility functions used by the Windows runner.
 - utils.h : This header file declares utility functions used across Windows runner modules.
 - win32_window.cpp : This file implements the native Win32 window logic for the Flutter application.
 - win32_window.h : This header file declares the Win32 window class and its methods.
- windows/runner/resources
 - app_icon.ico : This is the Windows application icon file used for the desktop executable.

D. Module 2 : Backend

1) Purpose

The backend of GomMold is designed to function as the central computational and data-management layer of the system. Its primary purpose is to coordinate communication between the mobile frontend, the AI-based mold detection model, and the cloud storage services. Implemented using Python and Flask, the backend is responsible for processing user requests, managing authentication workflows, and ensuring secure data transmission across all system components. It maintains the application’s operational integrity by storing user information, managing image data, and facilitating real-time interactions with the AI model. Deployed on Google Cloud, the backend provides a stable, scalable, and high-performance environment for

delivering the system's core services.

2) Functionality

The GomMold backend supports the full range of server-side operations required for the application. It manages user authentication through sign-up, login, token generation, and session validation mechanisms, ensuring controlled and secure access to system resources. The backend processes uploaded images, interacts with the AI model to generate mold detection results, and returns predictions including mold classification and confidence scores. Additionally, it integrates the OpenAI API to facilitate chatbot functionality, allowing users to obtain cleaning advice and mold-prevention guidance. The backend coordinates with Firebase Firestore and Google Cloud Storage to store user profiles, images, and detection history, enabling reliable data persistence and retrieval. Communication between modules is conducted through encrypted API endpoints, ensuring the confidentiality and integrity of all transmitted information. Overall, the backend provides the computational infrastructure necessary to support GomMold's intelligent, cloud-based mold detection ecosystem.

3) Location of source code : https://github.com/GomMold/GomMold_BackEnd

4) Class Components

- root
 - .gitignore : This file specifies which backend files and folders should be excluded from Git version control, such as virtual environments, cached files, and temporary outputs.
 - Procfile : Deployment configuration file that defines how the backend server should be launched.
 - README.md : Instructions on how to set up, run, and deploy the GomMold backend API.
 - requirements.txt : File listing all backend dependencies.
 - runtime.txt : File specifying the Python runtime version for deployment.
- src
 - __init__.py : Defines the backend package structure.
 - app.py : Main application file that runs the backend server and initializes routing.
 - firebase_init.py : Initializes Firebase Admin SDK for backend operations.
 - ML_model.py : Loads the trained ML model and performs prediction functions.
 - token_utils.py : Generates, validates, and manages JWT tokens for secure authentication.
- src/routes
 - __init__.py : Initializes the route package.
 - auth.py : Handles login, registration, and token-based authentication.

- chatbot.py : Manages chatbot queries and generates AI responses.
- history.py : Retrieves and manages user detection history.
- mold.py : Receives uploaded images, sends them to the ML model, and returns detection results.
- user.py : Handles user-related queries and profile interactions.
- src/models
 - best.onnx : A pre-trained mold detection model in ONNX format, used by the backend to perform inference on user-uploaded images.

E. Module 3 : AI

1) Purpose

The AI Module of GomMold serves as the core analytical engine responsible for the automated detection and classification of mold in user-submitted images. Implemented using a YOLO-based deep learning model in PyTorch, it ensures accurate, real-time recognition. The module encompasses data collection, preprocessing, model training, and fine-tuning to optimize performance under real-world conditions. Once trained, the model is deployed through the Flask backend, facilitating seamless communication with both frontend and backend components. Additionally, it integrates optional OpenAI-powered conversational capabilities to provide users with cleaning guidance and preventive advice. The primary objective of the AI Module is to deliver reliable, scalable, and efficient mold detection to support informed user decision-making and enhance overall application functionality.

2) Functionality

The GomMold AI Module receives images uploaded through the frontend, preprocesses them, and executes inference using the trained YOLO model to determine mold presence and associated confidence scores. Image datasets are annotated and prepared via Roboflow, trained in Google Colab, and deployed for real-time analysis within the backend. Detection results are transmitted back to the frontend through secure API endpoints, ensuring clear and interpretable outcomes for users. Furthermore, the module supports an optional OpenAI-powered chatbot that provides general cleaning tips, preventive guidance, and interactive follow-up responses. Overall, the AI Module ensures efficient AI-driven analysis, seamless integration with other system components, and an enhanced, data-informed user experience.

3) Custom Mold Detection Dataset (Roboflow – v4)

The dataset contains 1,134 annotated images of household surfaces.

- It consists of two classes:

- Mold
- No Mold
- Images were collected from:
 - Personal photography
 - Additional samples from public Roboflow Universe datasets
 - Additional samples from google
- Roboflow was used for:
 - Annotation
 - Dataset augmentation
 - Resizing all images to 640×640
 - Exporting to YOLOv8 format

4) System Configuration

The training was conducted using Google Colab GPU environment.

- Hardware (Colab) :
 - GPU: Tesla T4
 - RAM: 12–15GB
 - GPU VRAM: 16GB
- Software :
 - Python 3.10
 - PyTorch (CUDA-enabled)
 - Ultralytics YOLOv8
 - Roboflow SDK

5) Location of source code : https://github.com/GomMold/GomMold_ML

- root
 - README.md : The instructions for the overall repository
- models
 - README.md : Instructions for downloading the final best.pt model ((Model weights stored in GitHub Releases due to file size)
- training
 - GomMold_ML.ipynb : Main notebook for training the YOLOv8 model (Contains dataset loading, preprocessing, training commands, evaluation)
 - dataset_info.md : Full dataset documentation (version, structure, classes, augmentations)
- training/images
 - BoxF1_curve (1).png
 - BoxPR_curve (1).png
 - README.md
 - confusion_matrix (1).png

VII. USE CASE

A. Loading Page

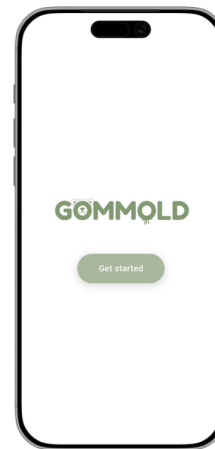


Fig. 21. Initial page

When users open the GomMold app, they are first greeted with a minimal welcome screen featuring the logo and a “Get Started” button. This page serves as the entry point of the application, guiding users smoothly into the main experience. From here, they can proceed to authentication or move directly into the app based on their login status.

B. Sign Up Page

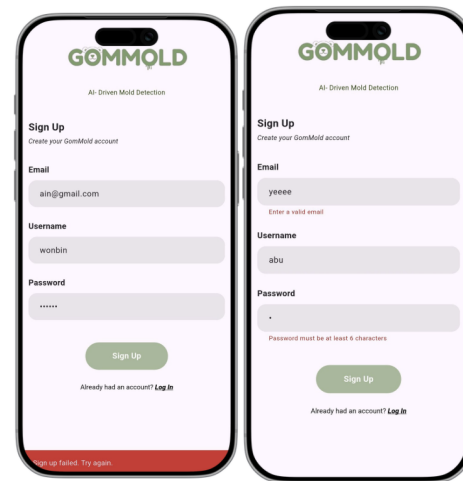


Fig. 22. Sign up error Fig. 23. Sign up error

When users enter incorrect or incomplete information on the Sign Up page, the app provides instant validation feedback by highlighting specific errors such as an invalid email format or a password that is too short. This allows users to correct their input before continuing. If the registration still fails due to invalid details or an email that already exists, the page displays an error message prompting the user to try again, ensuring a clear and guided sign-up process.

C. Login Page

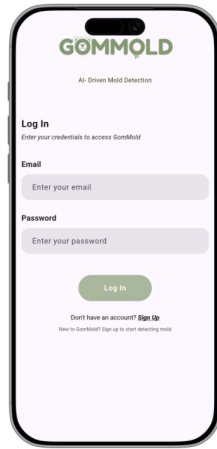


Fig. 24. Login Page

Users enter their email and password, and the system sends these credentials to the backend for verification. The backend validates the information against existing records, and if the credentials are correct, the user is authenticated and granted access to the application.

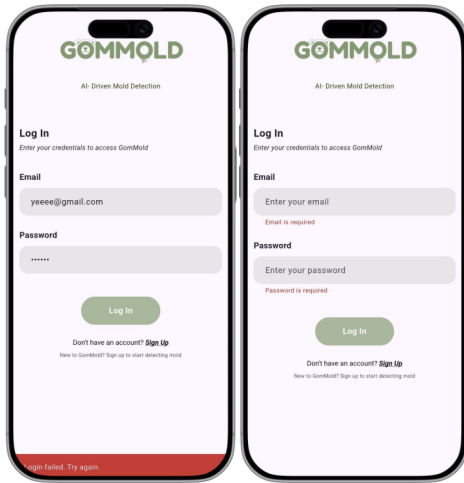


Fig. 25. Log in error

Fig. 26. Log in error

If users submit invalid or incomplete login information, the system immediately highlights the specific issues such as missing email or password, to ensure proper input before authentication. When valid fields are entered but the credentials do not match any existing account, the backend returns an error response, and the app displays a “Login failed” message, prompting the user to try again and recheck their details.

D. Main Page



Fig. 27. Main Page

The Main page displays all past mold detection results saved by the user. If no records are available, the page simply shows a “No history yet” message.

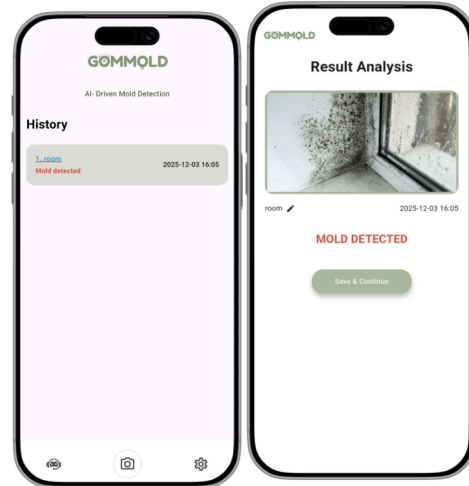


Fig. 28. History

Fig. 29. History information

Once detections exist, each entry appears with its name, detection status, and timestamp. When a user taps on any history item, the app opens the detailed Result Analysis page, allowing them to view the original image, detection outcome, and associated information exactly as it appeared during the initial analysis.

E. Image Page

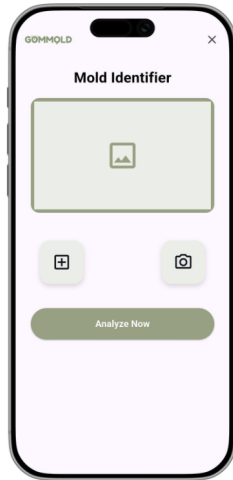


Fig. 30. Image Page

When users click the Camera icon or Photo icon, the app either requests camera access or opens the gallery, and once an image is captured or selected, it proceeds to the Identify page or shows an “Access to camera denied” message if permission is blocked.

F. Identify Page

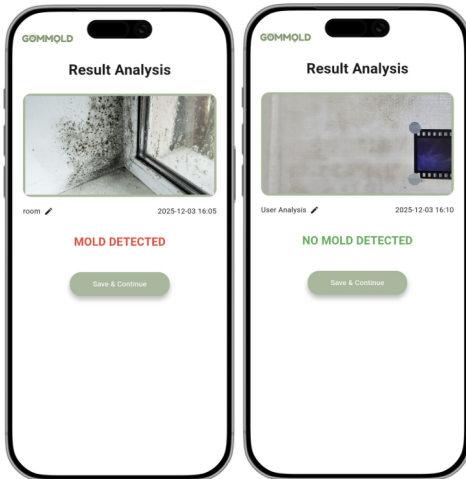


Fig. 31. Mold detected Fig. 32. No mold detected

When users take or upload a photo, the Identify page processes the image by sending it to the backend through a secured API request. The backend loads the ONNX-based YOLO detection model, runs inference on the image, and returns a structured JSON response containing the prediction result. The Identify page then interprets this response and displays the appropriate outcome such as a mold detected alert or a safe message. At the same time, the system automatically stores the image URL, detection name, timestamp, and model output in Firestore, ensuring that every analysis is recorded and can be accessed later through the History page.

G. Chatbot Page

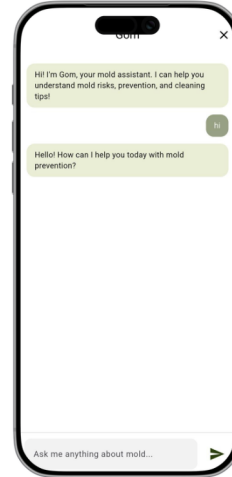


Fig. 33. Chatbot page

The Chatbot page allows users to interact with Gom, an AI-powered assistant designed to answer questions about mold detection, prevention, and cleaning. When a user submits a message, the app sends the query to the backend, which forwards it to the OpenAI API for natural language processing. The API generates an appropriate response based on the user's input, and the backend returns this message to the app, where it is displayed in the chat interface. This creates a real-time conversational experience that helps users understand mold-related issues and receive guidance directly within the application.

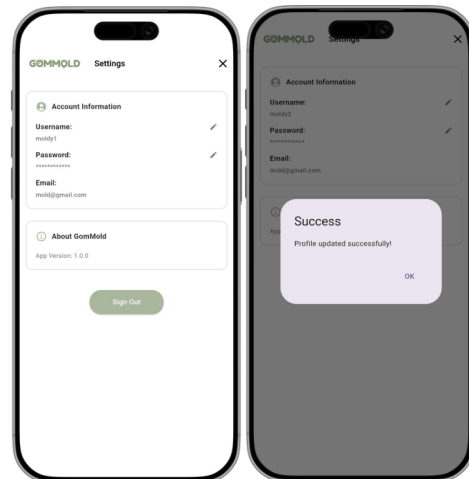


Fig. 34. Settings page Fig. 35. Settings page

The Settings page allows users to view and manage their account information, including their username and password, all of which are retrieved from the backend upon login. When a user chooses to update their username or password, the app opens an edit dialog where the new value can be entered. After the user submits the change, the app sends an update

request to the backend, which validates the input and writes the updated data to Firestore. Once the backend confirms a successful update, the app displays a success message and refreshes the account information on the page.

VIII. DISCUSSION

Throughout the development of GomMold, our team faced a wide range of challenges that affected the backend, AI/ML model, and frontend components equally. On the backend side, the most difficult aspect was deploying the server to the cloud. Although our original plan was to use Google Cloud because of our reliance on Firebase, the deployment process quickly became overwhelming. Build times were extremely long, and deployments repeatedly failed with unclear error messages that were buried in long, complex logs. Troubleshooting was especially difficult because GCP relies on many layers of configuration, such as Docker images, service accounts, IAM permissions, and runtime settings. Even after numerous attempts to fix the issues, each redeployment introduced new problems, making the process discouraging and time-consuming. We then shifted to Render, which simplified the interface but created new complications, especially Python version mismatches and model loading failures during the build stage. Render's requirement to redeploy through GitHub also led to many experimental commits. Eventually, we switched to Railway, which provided faster builds and clearer error logs. However, even Railway required several adjustments, especially with library compatibility and model-loading logic. Integrating the backend with the frontend introduced another layer of work, as API responses and field names had to be aligned precisely to avoid UI errors.

The AI/ML component was equally challenging. Collecting a high-quality mold dataset required much more effort than we expected. Online images were limited, copyrighted, or inconsistent in quality, and the photos we took ourselves lacked variety in mold types and lighting conditions. Annotating these images was tedious because mold does not have sharp boundaries, meaning every small patch had to be carefully labeled. Even minor annotation mistakes reduced the model's accuracy, causing confusion between mold, stains, and shadows. Initially, we planned to classify mold into different colors, but the dataset was too small and inconsistent to train a reliable color-based model. We also considered classifying mold by species or type, but this required expert knowledge and specialized datasets that we did not have access to. After training, the PyTorch model performed well locally but caused deployment errors on multiple cloud platforms. This forced us to convert the model to ONNX, which introduced a new set of challenges. The ONNX model produced different output structures, tensor shapes, and confidence values compared to the PyTorch version. We had to rewrite much of our post-processing code and re-export the model multiple times before the outputs became stable. Integrating the ONNX model with the backend required detailed testing, especially when sending results to the frontend, ensuring that

color labels, messages, and analysis names appeared correctly.

On the frontend, the team faced both technical and communication-related challenges. Because different members worked on different devices, repository synchronization became a recurring problem. Some cloned versions were outdated, certain pages were missing after pulls, and duplicated folders caused confusion about the latest build. Flutter's routing system also required careful attention, linking UI buttons to their correct pages and maintaining a smooth navigation flow needed multiple revisions. Ensuring consistent design across devices was another challenge, as Flutter's layout system reacts differently depending on screen size, which led to spacing issues, misaligned components, and distorted images. The frontend also frequently broke whenever backend API outputs changed. For example, a slight change in the JSON format could cause the history page to display empty results or load incorrect images. Fixing these mismatches required constant communication and collaboration between frontend and backend members.

Overall, the combination of backend deployment issues, AI model preprocessing difficulties, and frontend synchronization problems made this project a complex but meaningful learning experience. Despite the challenges, the project helped us build confidence in cloud deployment, dataset preparation, machine-learning integration, version control, and cross-team communication. Since this was our first large-scale development project, much of our time was spent learning by doing, which naturally led to mistakes but also to significant growth. Moving forward, we plan to adopt clearer communication methods, establish a more structured Git workflow, and perform regular code reviews to ensure smoother integration. These improvements will help us work more efficiently in future projects and avoid many of the issues we encountered during GomMold's development.

REFERENCES

- [1] App Store. (n.d.). Mould Identifier & AI Scanner App. <https://apps.apple.com/kr/app/mould-identifier-ai-scanner/id6743941888?l=en-GB>
- [2] App Store. (n.d.). Mold Finder AI, Inspection App. <https://apps.apple.com/kr/app/mold-finder-ai-inspection/id6689505239?l=en-GB>
- [3] AND Academy. Stevens, E. (2025, August 14). What is Figma – uses, benefits, and features. <https://www.andacademy.com/resources/blog/graphic-design/what-is-figma/>
- [4] Back4App Blog. Team, B. E. (2021, September 9). An overview of Firebase BaaS. <https://blog.back4app.com/backend-as-a-service-firebase/>
- [5] Bergmann, D. (2025, October 23). What is machine learning? IBM. <https://www.ibm.com/think/topics/machine-learning>
- [6] Bergmann, D., & Stryker, C. (2025, October 20). What is PyTorch? IBM. <https://www.ibm.com/think/topics/pytorch>
- [7] Build apps for any screen. Flutter. (n.d.). <https://flutter.dev/>
- [8] Dart Programming Language: Top features and applications. Khan, S. (2025, October 17). Xavor Corporation. <https://www.xavor.com/blog/dart-programming-language-features-and-applications/>
- [9] EBSCO Research Starters. (n.d.). Windows operating system. <https://www.ebsco.com/research-starters/computer-science/windows-operating-system>

- [10] Expert Software Development — Kaopiz Holdings. Hng, H. N. (2025, May 26). What is OpenAI? <https://kaopiz.com/en/articles/what-is-openai/>
- [11] Firebase. Google. (n.d.). <https://firebase.google.com/>
- [12] Flutter Architectural Overview. Flutter Docs. (n.d.-b). <https://docs.flutter.dev/resources/architectural-overview>
- [13] Flask Documentation. Welcome to Flask (3.1.x). <https://flask.palletsprojects.com/en/stable/>
- [14] Fox, L. (2023, September 17). Why should you use Flask Framework for web development? Medium. <https://medium.com/@lauren-fox/why-should-you-use-flask-framework-for-web-development-f5a7233e17a6>
- [15] GitHub Introduction. Holcombe, J. (2025, August 28). Kinsta. <https://kinsta.com/blog/what-is-github/>
- [16] Google Cloud Overview. Google. (n.d.-a). <https://cloud.google.com/docs/overview>
- [17] Google. (n.d.). Add the Firebase Admin SDK to your server. <https://firebase.google.com/docs/admin/setup>
- [18] Hot Reload. Flutter Docs. (n.d.-c). <https://docs.flutter.dev/tools/hot-reload>
- [19] IBM. Gillis, A. S. (2025, June 11). What is macOS? WhatIs. <https://www.techtarget.com/whatis/definition/Mac-OS>
- [20] ITarian Blog. (n.d.). What is macOS? Features, Benefits & Latest Version. <https://www.itarian.com/blog/what-is-mac-os/>
- [21] Kaopiz. Hng, H. N. (2025, May 26). What is OpenAI? <https://kaopiz.com/en/articles/what-is-openai/>
- [22] Microsoft. (2021, November 3). Visual Studio Code – Code editing, redefined. <https://code.visualstudio.com/>
- [23] Mold Detection Dataset. Roboflow Universe. <https://universe.roboflow.com/dissertationproject/mould-detection-aaron>
- [24] Mold Identifier: AI Scan App. Worlitzer, M. (n.d.). App Store. <https://apps.apple.com/kr/app/mold-identifier-ai-scan/id6749549375?l=en-GB>
- [25] MoldAI App. Y. P. C. Limited. (n.d.). App Store. <https://apps.apple.com/kr/app/moldai/id6689520833?l=en-GB>
- [26] Nguyen, A. (2025, May 5). Flask Framework: Why developers love it for web projects. Stepmedia Software. <https://stepmediasoftware.com/blog/what-is-flask-used-for/>
- [27] Notion. (n.d.). The all-in-one workspace for your notes, tasks, wikis, and databases. <https://www.notion.so/index.html>
- [28] Notion AI Workspace. (n.d.-a). <https://www.notion.so/notion/What-s-New-157765353f2c4705bd45474e5ba8b46c>
- [29] Overleaf. Online LaTeX Editor. (n.d.). <https://www.overleaf.com/>
- [30] Patel, B. (2025, May 29). Top 10 benefits of using OpenAI in web app development. Space-O Technologies. <https://www.spaceotechnologies.com/blog/benefits-of-using-openai-web-app-development/>
- [31] Postman. (n.d.). The world's leading API platform. <https://www.postman.com/>
- [32] PyTorch. (n.d.). <https://pytorch.org/>
- [33] Python.org. (n.d.). What is Python? <https://www.python.org/doc/essays/blurb/>
- [34] Railway Docs. (n.d.). <https://docs.railway.com/>
- [35] Roboflow Universe. Wall Computer Vision Dataset. <https://universe.roboflow.com/jisupark/wall-yqazk>
- [36] Safi, A. (n.d.). Understanding BaaS: Firebase vs. Supabase. <https://abdulkadersafi.com/blog/understanding-baas-a-deep-dive-into-firebase-vs-supabase>
- [37] Ultralytics YOLO Docs. (2025, November 23). <https://docs.ultralytics.com/>
- [38] Windows. Digital Unite. <https://www.digitalunite.com/technology-guides/computer-essentials/windows-10/what-windows>